Faster deterministic Feedback Vertex Set*

Tomasz Kociumaka† Marcin Pilipczuk‡

Abstract

We present two new deterministic algorithms for the Feedback Vertex Set problem parameterized by the solution size. We begin with a simple algorithm, which runs in $\mathcal{O}^*((2+\phi)^k)$ time, where $\phi < 1.619$ is the golden ratio. It already surpasses the previously fastest $\mathcal{O}^*((1+2\sqrt{2})^k)$-time deterministic algorithm due to Cao et al. [SWAT 2010]. In our developments we follow the approach of Cao et al., however, thanks to a new reduction rule, we obtain not only better dependency on the parameter in the running time, but also a solution with simple analysis and only a single branching rule. Then, we present a modification of the algorithm which, using a more involved set of branching rules, achieves $\mathcal{O}^*(3.592^k)$ running time.

1 Introduction

The Feedback Vertex Set problem (FVS for short), where we ask to delete as few vertices as possible from a given undirected graph to make it acyclic, is one of the fundamental graph problems, appearing on the Karp's list of 21 NP-hard problems [21]. Little surprise it is also one of the most-studied problems in parameterized complexity, and a long race for the fastest FPT algorithm (parameterized by the solution size, denoted k) includes [2, 15, 16, 23, 20, 14, 19, 7, 6, 1, 12]. Prior to this work, the fastest *deterministic* algorithm, due to Cao et al. [6], runs in $\mathcal{O}^*((1+2\sqrt{2})^k) \leq \mathcal{O}^*(3.83^k)$ time[1]; if we allow randomization, the Cut&Count technique yields an $\mathcal{O}^*(3^k)$-time algorithm [12]. Further research investigates kernelization complexity of FVS [5, 4, 25] and some generalizations e.g. to directed graphs [8, 13, 9].

In this work we claim the lead in the 'FPT race' for the fastest *deterministic* algorithm for FVS.

Theorem 1. Feedback Vertex Set*, parameterized by the solution size k, can be solved in $\mathcal{O}^*(3.592^k)$ time and polynomial space.*

First, we present much simpler algorithm which proves a slightly weaker result.

Theorem 2. Feedback Vertex Set*, parameterized by the solution size k, can be solved in $\mathcal{O}^*((2+\phi)^k) \leq \mathcal{O}^*(3.619^k)$ time and polynomial space where $\phi = \frac{1+\sqrt{5}}{2} < 1.619$ is the golden ratio.*

In our developments, we closely follow the approach of the previously fastest algorithm due to Cao et al. [6]. That is, we first employ the iterative compression principle [24] in a standard manner to reduce the problem to the disjoint compression variant (Disjoint-FVS), where the vertex set is split into two parts, both inducing forests, and we are allowed to delete vertices only from the second part. Then we develop a set of reduction and branching rule(s) to cope with this structuralized instance. We rely on the core observation of [6] that the problem becomes polynomial-time solvable once the maximum degree of the deletable vertices drops to 3.

The main difference between our algorithm and the one of [6] is the introduction of a new reduction rule that reduces deletable vertices with exactly one deletable neighbour and two undeletable ones. Branching on such vertices is the most costly operation in the $\mathcal{O}^*(5^k)$ algorithm of Chen et al. [7] and avoiding such branching is a source of some complications in the algorithm of [6]. Introducing the new rule allows us to perform later only a single straightforward branching rule. Hence, the new rule not only leads to a better time complexity, but also allows us to simplify the algorithm and analysis, comparing to [6].

*Partially supported by NCN grant UMO-2012/05/D/ST6/03214 and Foundation for Polish Science.

†Institute of Informatics, University of Warsaw, Poland, kociumaka@mimuw.edu.pl

‡Institute of Informatics, University of Warsaw, Poland, malcin@mimuw.edu.pl

[1]The $\mathcal{O}^*$-notation suppresses factors polynomial in the input size.

Additionally, we present a new, shorter proof of the main technical contribution of [6], asserting that Disjoint-FVS is polynomial-time solvable if all deletable vertices are of degree at most 3. Thus, apart from better time complexity, we contribute a simplification of the arguments of Cao et al. [6].

Finally, we modify our algorithm so that it achieves slightly better time complexity. This requires changes in the instance measure as well as introducing several branching rules. The branching rules use each other as subroutines, which makes their branching vectors long. Instead of manually determining this branching vectors, we provide a script, which contains a compact and easily readable representation of the branching rules and based on this data determines the complexity of our algorithm.

1.1 Preliminaries and notation

All graphs in our work are undirected and, unless explicitly specified, simple. For a graph G, by $V(G)$ and $E(G)$ we denote its vertex- and edge-set, respectively. For $v \in V(G)$, the neighbourhood of v, denoted $N_G(v)$, is defined as $N_G(v) = \{u \in V(G) : uv \in E(G)\}$. For a set $X \subseteq V(G)$, we denote the subgraph induced by X as $G[X]$ and $G \setminus X$ denotes $G[V(G) \setminus X]$. For $X \subseteq V(G)$ and $v \in V(G)$ we define an X-degree of v, denoted by $\deg_X(v)$, as $|N_G(v) \cap X|$. Moreover, for $v \in X$ we say that v is X-isolated if $\deg_X(v) = 0$, and an X-leaf if $\deg_X(v) = 1$.

If uv is an edge in a (multi)graph G, by *contracting the edge* uv we mean the following operation: we replace u and v with a new vertex x_{uv}, introduce $p-1$ loops at x_{uv}, where p is the multiplicity of uv in G, and, for each $w \in (N_G(u) \cup N_G(v)) \setminus \{u, v\}$, introduce an edge wx_{uv} of multiplicity equal to the sum of the multiplicities of wu and wv in G. In other words, we do not suppress multiple edges and loops in the process of contraction. Note that, if G is a simple graph and $N_G(u) \cap N_G(v) = \emptyset$, no loop nor multiple edge is introduced when contracting uv.

2 The simple algorithm

2.1 Iterative compression

Following [6], we employ the iterative compression principle [24] in a standard manner. Consider the following variant of FVS.

Disjoint-FVS
Input: Graph G, a partition $V(G) = U \cup D$ such that both $G[U]$ and $G[D]$ are forests, and an integer k.
Question: Does there exist a set $X \subseteq D$ of size at most k such that $G \setminus X$ is a forest?

A D-isolated vertex of degree 3 is called a *tent*. For a Disjoint-FVS instance $I = (G, U, D, k)$ we define the following invariants: $k(I) = k$, $\ell(I)$ is the number of connected components of $G[U]$, $t(I)$ is the number of tents in I, and $\mu(I) = k(I) + \ell(I) - t(I)$ is the *measure* of I. Note that our measure differs from the one used in [6]. We omit the argument if the instance is clear from the context.

In the rest of the paper we focus on solving Disjoint-FVS, proving the following theorem.

Theorem 3. Disjoint-FVS *on an instance I can be solved in $\mathcal{O}^*(\phi^{\max(0,\mu(I))})$ time and polynomial space.*

For sake of completeness, we show how Theorem 3 implies Theorem 2.

Proof of Theorem 2. Assume we are given a FVS instance (G, k). Let $v_1, \ldots, v_n$ be an arbitrary ordering of $V(G)$. Define $V_i = \{v_1, v_2, \ldots, v_i\}$, $G_i = G[V_i]$; we iteratively solve FVS instances (G_i, k) for $i = 1, 2, \ldots, n$. Clearly, if (G_i, k) turns out to be a NO-instance for some i, (G, k) is a NO-instance as well. On the other hand, (G_i, k) is a trivial YES-instance for $i \leq k + 1$.

To finish the proof we need to show how, given a solution X_{i-1} to (G_{i-1}, k), solve the instance (G_i, k). Let $Z := X_{i-1} \cup \{v_i\}$ and $D = V_i \setminus Z = V_{i-1} \setminus X_{i-1}$. Clearly, $G[D]$ is a forest. We branch into $2^{|Z|} \leq 2^{k+1}$ subcases, guessing the intersection of the solution to (G_i, k) with the set Z. In a branch labeled $Y \subseteq Z$, we delete Y from G_i and disallow deleting vertices of $Z \setminus Y$. More formally, for any $Y \subseteq Z$ such that $G_i[Z \setminus Y]$ is a forest, we define $U = Z \setminus Y$ and apply the algorithm of Theorem 3 to the Disjoint-FVS instance $I_Y = (G_i \setminus Y, U, D, k - |Y|)$. Clearly, I_Y is a YES-instance to Disjoint-FVS iff (G_i, k) has a solution X_i with $X_i \cap Z = Y$.

As for the running time, note that $\ell(I_Y) < |U| = |Z \setminus Y| \leq (k+1) - |Y|$. Hence, $\mu(I_Y) \leq 2(k - |Y|)$ and the total running time of solving (G_i, k) is bounded by

$$\mathcal{O}^* \left(\sum_{Y \subseteq Z} \phi^{2(k-|Y|)} \right) = \mathcal{O}^* \left((1+\phi^2)^k \right) = \mathcal{O}^* \left((2+\phi)^k \right).$$

This finishes the proof of Theorem 2. □

2.2 Reduction rules

Assume we are given na Disjoint-FVS instance $I = (G, U, D, k)$. We first recall the (slightly modified) reduction rules of [6]. At any time, we apply the lowest-numbered applicable rule.

Reduction Rule 1. Remove all vertices of degree at most 1 from G.

Reduction Rule 2. If a vertex $v \in D$ has at least two neighbours in the same connected component of $G[U]$, delete v and decrease k by one.

Reduction Rule 3. If there exists a vertex $v \in D$ of degree 2 in G, move it to U if it has a neighbour in U, and contract one of its incident edges otherwise.

We shortly discuss the differences in the statements between our work and [6]. We say that a rule is *safe* if the output instance is a YES-instance iff the input one is. First, we apply Rule 2 to any vertex with many neighbours in the same connected component of $G[U]$, not only to the degree-2 ones; however, the safeness of the new rule is straightforward, as any solution to Disjoint-FVS on I needs to contain such v. Second, we prefer to contract an edge in Reduction 3 in case when $N_G(v) \subseteq D$, instead of moving v to U, to avoid an increase of the measure $\mu(I)$. Note that, as G is simple and $G[D]$ is a forest, no multiple edge is introduced in such contraction and G remains a simple graph.

By [6] and the argumentation above we infer that Rules 1–3 are safe and applicable in polynomial time. Let us now verify the following.

Lemma 4. *An application of any of the Rules 1–3 does not increase $\mu(I)$.*

Proof. Consider first an application of Rule 1 to a vertex v. If $v \in D$, $k(I)$, $\ell(I)$ and $t(I)$ remains unchanged, as v is not a tent. If $v \in U$, $k(I)$ remains unchanged and $\ell(I)$ does not increase. Note that $t(I)$ may decrease if the sole neighbour of v is a tent. However, in this case $\ell(I)$ also drops by one, and $\mu(I)$ remains unchanged.

If Rule 2 is applied to v, $k(I)$ drops by one, $\ell(I)$ remains unchanged and $t(I)$ remains the same or drops by one, depending on whether v is a tent or not.

If Rule 3 is applied to v, $k(I)$ remains unchanged, $t(I)$ does not decrease and $\ell(I)$ does not increase (thanks to the special case of $N_G(v) \subseteq D$). □

We now prove the lower bound on the measure $\mu(I)$ (cf. [6], Lemma 1):

Lemma 5. *Let I be a* Disjoint-FVS *instance. If $t(I) \geq k(I) + \frac{1}{2}\ell(I)$, then I is a NO-instance.*

Proof. Assume $I = (G, U, D, k)$ is a YES-instance and let X be a solution. Let $T \subseteq D$ be a set of tents in I. For any $v \in T \setminus X$, v connects three connected components of $G[U]$, hence $2|T \setminus X| < \ell(I)$. As $|X| \leq k$, $|T| < k + \frac{1}{2}\ell(I)$ and the lemma follows. □

Consequently, we may apply the following rule.

Reduction Rule 4. If $\mu(I) \leq \frac{1}{2}\ell(I)$, conclude that I is a NO-instance.

Note that Rule 4 triggers when $\mu(I) \leq 0$.

We now introduce a new reduction rule, promised in the introduction.

Reduction Rule 5. If v is a D-leaf of U-degree 2 with w being its only neighbour in D, subdivide the edge vw and insert the newly created vertex to U.

First, we note that Rule 5 is safe: introducing an undeletable degree-2 vertex in a middle of an edge does not change the set of feasible solutions to Disjoint-FVS. Second, note that Rule 5 does not increase the measure of the instance: although the newly added vertex creates a new connected component in $G[U]$, increasing $\ell(I)$ by one, at the same time v becomes a tent and $t(I)$ increases by at least one (w may become a tent as well). Third, one may be worried that Rule 5 in facts expands G, introducing a new vertex. However, it also increases the number of tents; as our algorithm never inserts a vertex to D, Rule 5 may be applied at most $|D|$ times.

2.3 Polynomial-time solvable case

Before we move to the branching rule, let us recall the polynomial-time solvable case of Cao et al. [6].

Theorem 6 ([6])**.** *There exists a polynomial-time algorithm that solves a special case of* Disjoint-FVS *where each vertex of D is a tent.*

Strictly speaking, in [6] a seemingly more general case is considered where each vertex of D is of degree exactly three in G. However, it is easy to see that an exhaustive application of Rule 5 to such an instance results in an instance where each vertex of D is a tent.

Theorem 6 allows us to state the last reduction rule.

Reduction Rule 6. If each vertex of D is a tent, resolve the instance in polynomial time using the algorithm of Theorem 6.

Theorem 6 is proven in [6] by a reduction to the matroid parity problem in a cographic matroid. We present here a shorter proof, relying on the matroid parity problem in a graphic matroid of some contraction of G.

Given a (multi)graph H, the *graphic matroid* of H is a matroid with ground set $E(H)$ where a set $A \subseteq E(H)$ is independent iff A is acyclic in H (spans a forest)[2].

The *matroid parity problem* on the graphic matroid H is defined as follows. In the input, apart from the (multi)graph H with even number of edges, we are given a partition of $E(H)$ into pairs; in other words, $|E(H)| = 2m$ and $E(H) = \{e_1^1, e_1^2, e_2^1, e_2^2, \ldots, e_m^1, e_m^2\}$. We seek for a maximum set $J \subseteq \{1, 2, \ldots, m\}$ such that $A(J) := \bigcup_{j \in J} \{e_j^1, e_j^2\}$ is independent, i.e., is acyclic in H. The matroid parity problem in graphic matroids is polynomial-time solvable [18].

Having introduced the matroid parity problem in graphic matroids, we are ready to present a shorter proof of Theorem 6.

Proof of Theorem 6. Let $I = (G, D, U, k)$ be a Disjoint-FVS instance where each vertex of D is a tent. For each $v \in D$ we arbitrarily enumerate the edges incident to v as e_v^0, e_v^1, e_v^2. Define $S = E(G[U]) \cup \{e_v^0 : v \in D\}$ and let H be the multigraph obtained from G by contracting all edges of S (recall that we do not suppress the multiple edges and loops in the process of contraction). Clearly, $E(H) = \{e_v^1, e_v^2 : v \in D\}$. Treat H, with pairs $\{e_v^1, e_v^2\}_{v \in D}$ as an input to the matroid parity problem in the graphic matroid of H.

Let $J \subseteq D$. We claim that $A(J) = \bigcup_{v \in J} \{e_v^1, e_v^2\}$ is independent in the graphic matroid in H iff $G \setminus (D \setminus J)$ is a forest. Note this claim finishes the proof of Theorem 6 as it implies that a maximum solution J to the matroid parity problem corresponds to a minimum solution to Disjoint-FVS on I.

First note that, by the definition of H, $A(J)$ is acyclic in H iff $S \cup A(J)$ is acyclic in G. Hence, if $A(J)$ is acyclic in H, $G \setminus (D \setminus J)$ is a forest, as $E(G \setminus (D \setminus J)) \subseteq S \cup A(J)$. In the other direction, assume that $G \setminus (D \setminus J)$ is acyclic. Note that

$$S \cup A(J) = E(G \setminus (D \setminus J)) \uplus \{e_v^0 : v \in D \setminus J\}.$$

However, each vertex $v \in D \setminus J$ has only one incident edge that belongs to $S \cup A(J)$. Hence, $S \cup A(J)$ is acyclic and the claim is proven. □

[2]For basic definitions and results on matroids and graphic matroids, we refer to the monograph [22].

2.4 Branching rule

We conclude with a final branching rule.

Branching Rule 7. Pick a vertex $v \in D$ that is not a tent and has a maximum possible number of neighbours in U. Branch on v: either delete v and decrease k by one or move v to U.

First, note that the branching of Rule 7 is exhaustive: in first branch, we consider the case when v is included in the solution to the instance I we seek for, and in the second branch — when the solution does not contain v.

In the branch where v is removed, the number of tents does not decrease and the number of connected components of $G[U]$ remains the same. Hence, the measure $\mu(I)$ drops by at least one.

Let us now consider the second branch. As Rule 6 is not applicable, there exists a connected component of $G[D]$ that is not a tent, and there exists a vertex u in this component whose degree in $G[D]$ is at most one. As Rules 1, 3 and 5 are not applicable, u has at least three neighbours in U. As u is not a tent, Rule 7 may choose u and, consequently, it triggers on a vertex v with at least 3 neighbours in U. As Rule 2 is not applicable, in the branch where v is moved to U, $G[U]$ remains a forest and the number of its connected components, $\ell(I)$, drops by at least two. Hence, as $t(I)$ does not decrease and $k(I)$ remains unchanged in this branch, the measure $\mu(I)$ drops by at least two.

By Lemma 4, no reduction rule may increase the measure $\mu(I)$. Rule 4 terminates computation if $\mu(I)$ is not positive. The branching rule, Rule 7, yields drops of measure 1 and 2 in the two considered subcases. We conclude that the algorithm resolves an instance I in $\mathcal{O}^*(\phi^{\mu(I)})$ time and polynomial space, concluding the proof of Theorem 3.

3 The $\mathcal{O}^*(3.592^k)$-time algorithm

In the improved algorithm we still follow the iterative compression principle and work with Disjoint-FVS problem.

Let $I = (G, D, U, k)$ be an instance of Disjoint-FVS. For technical reasons we no longer require that $G[U]$ is forest, despite the fact that whenever $G[U]$ contains a cycle, I is clearly a NO-instance. Let us define $\ell'(I) = |U| - |E(G[U])|$. Note that $\ell'(I)$ generalizes $\ell(I)$ defined for instance with acyclic $G[U]$ as $\ell'(I)$ equals then the number of connected components of $G[U]$.

For a real constant $\alpha \in \left[\frac{1}{2}, 1\right]$ we define $\mu_\alpha(I) = k(I) + \alpha\ell'(I) - t(I)$. Recall that the previous algorithm used $\alpha = 1$ while in [6] this parameter is set to $\frac{1}{2}$.

3.1 Reduction rules

We use reduction rules similar to those defined Section 2.2. Since we use a more general measure and some of the rules differ, we explicitly state all the rules we use and briefly discuss their correctness.

Reduction Rule 1. Remove all vertices $v \in D$ of degree at most 1.

Reduction Rule 2. If there exists a vertex $v \in D$ of degree 2, move it to U if it has a neighbour in U, and contract one of its incident edges otherwise.

Reduction Rule 3. If $\mu_\alpha(I) \leq (\alpha - \frac{1}{2})\ell'(I)$, conclude that I is a NO-instance.

Reduction Rule 4. If v is a D-leaf of U-degree 2 with w being its only neighbour in D, subdivide the edge vw and insert the newly created vertex to U.

Reduction Rule 5. If each $v \in D$ is a tent, resolve the instance in polynomial time using the algorithm of Theorem 6 if $G[U]$ is a forest, and return NO otherwise.

Lemma 7. *Rules 1–5 are safe. Moreover, each of them either immediately gives the answer or decreases $|D|$ simultaneously not increasing the measure μ_α*

Proof. All rules except Rule 3 are safe since their counterparts in Section 2.2 were safe. To prove the safeness of Rule 3 observe that if $G[U]$ is a forest, then the instance is trivially a NO-instance and

otherwise $\ell(I)$ coincides with $\ell'(I)$ so Lemma 5 immediately shows that I is a NO-instance if the rule is applicable.

Now, let us analyze the change in measure. Rule 1 leaves k and ℓ' unchanged while t may only increase since the neighbour of the vertex removed from D might become a tent. Rule 2 leaves k unchanged, and depending on the case, either does not modify $G[U]$ or introduces one vertex and one or two edges to $G[U]$. Consequently, ℓ' does not increase. The rule may create a tent, but does not remove any, so t does not decrease. Rule 4 introduces one vertex to $G[U]$, so it increases ℓ' by one. Simultaneously, it leaves k unchanged and introduces at least one tent. In total, this gives a drop of at least $(1-\alpha)$ in the measure. □

We apply the reduction rules in any order until we obtain an *irreducible* instance, i.e. no rule is applicable. Let us gather some properties of such instances.

Lemma 8. *Assume that $I = (G, U, D, k)$ is an irreducible instance. Then*

(a) each $v \in D$ of U-degree 0 has D-degree at least 3,

(b) $\mu_\alpha(I) > 0$,

(c) there is at least one $v \in D$ which is not a tent,

(d) each D-leaf and D-isolated vertex has U-degree 3 or more.

Proof. Property (a) is an immediate consequence of inapplicability of Rules 1 and 2, property (b) of Rule 3 and property (c) of Rule 5. For a proof of (d) observe that D-isolated vertices of degree 0 and 1 are eliminated by Rule 1, while of degree 2 by Rule 2. Similarly, D-leaves of U degree 0,1 and 2 are dismissed by Rules 1, 2 and 4 respectively. □

3.2 Branching rules

If no reduction rules is applicable, the algorithm performs one of the branching rules. The branching rules are built from several elementary operations. These operations include performing Rules 1, 2 or 4 as well as the following branching rule:

Branching Rule 6 (Branching on v)**.** Let $v \in D$. Either delete v and decrease k by one (*delete* branch) or move v to U (*fix* branch).

Similarly to the branching rule of the previous algorithm, this rule is clearly exhaustive. Observe that if v is not a tent, then in the *delete* branch the measure decreases by at least one (since k decreases) and in the *fix* branch, the measure decreases by at least $\alpha(f-1)$, where f is the U-degree of v. This is because one vertex and f edges are moved to $G[U]$. Also, if v has a parent, its U-degree remains unchanged in the *delete* branch and raises by one in the *fix* branch.

With all building blocks ready, let us proceed with the description of the structure steering execution of the algorithm. For each connected component T of $V[D]$ we select one of the vertices of T as the *root* of T. We require the root to be either D-isolated or a D-leaf. The roots are selected every time we need to choose a branching rule, each such selection can be performed independently. With a root in each component, we can define the parent-child relation on D, which we use to partition vertices of D into four types:

(a) *tents*

(b) other vertices of U-degree 3 with no children, called *singles*,

(c) vertices of U-degree 0 with two children, both singles, called *doubles*,

(d) the remaining vertices, called *standard* vertices.

We call a standard vertex a *guide* if none of its children is standard. Each guide has three parameters: the U-degree f, the number of children being singles s, and the number of children being doubles d. We call the triple (f, s, d) the *type* of a guide. In all branching rules we pick an arbitrary guide v and perform operations on the vertices in a subtree rooted on v. The choice of the branching rule depends on the type of the guide. The following lemma justifies such an approach.

Lemma 9. *If $I = (G, U, D, k)$ is irreducible, D contains a guide.*

Proof. Clearly, it suffices to prove that there exists a standard vertex in D. Observe that if v is standard, so is its parent (if any). Consequently, we shall prove that a root of some component of $G[D]$ is a standard vertex. By Lemma 8(c) there is a vertex $v \in D$, which is not a tent. Let T be its connected component of $G[D]$ and r be a root of T. Note that, since roots were chosen to have D-degree at most 1, by Lemma 8(c) r has U-degree at least 3. Thus, r is not a double. If r were a single, it would be D-isolated, then, however, it would either be a tent or its U-degree would be at least 4. The former case is impossible by the choice of T while in the latter r is standard as desired. Therefore r is standard. □

Finally, observe that some triples of non-negative integers cannot be types of a guide. Indeed, types $(0, 0, 1)$ and $(0, 1, 0)$ contradict Lemma 8(a), while types $(0, 0, 0)$, $(1, 0, 0)$ and $(2, 0, 0)$ Lemma 8(d). Moreover, types $(3, 0, 0)$ and $(0, 2, 0)$ are forbidden, since a guide of this type would actually be a single, tent or a double. Also note that, since any root has U-degree at least 3, a guide with $f \leq 2$ always has a parent.

Let us start with a pair of rules, which are used as subroutines in other rules. All branching rules are indexed by guide types. We use $\geq n$ as 'at least n'. If a guide matches several rules, any of them can be applied. In all the descriptions the guide is called v.

Branching Rule $(\geq 4, 0, 0)$**.** Branch on v (i.e. perform Rule 6).

Branching Rule $(1, 1, 0)$**.** Branch on w, the only child of v. In the *delete* branch, use Rule 2 to move v to U. In the *fix* branch, use Rule 4 to make v a tent. In both branches, the U-degree of a parent of v raises by one.

Before we list the remaining rules, let us describe a subroutine which is not used as a branching rule on its own.

Subroutine (Eliminate a double)**.** Let v be a guide of type (f, s, d) and let its child u be a double with children w_1, w_2. Branch on w_1. In the *delete* branch, use Rule 2 to dissolve u. Then, v becomes an $(f, s + 1, d - 1)$-guide. In the *fix* branch, u becomes a $(1, 1, 0)$-guide, proceed with the $(1, 1, 0)$ rule. In both branches of this rule, v becomes an $(f + 1, s, d - 1)$-guide.

Below, we give branching rules for all types of guides not considered yet. Most of these rules use the same scheme, but for sake of clarity and consistence with complexity analysis, we state all rules explicitly.

Branching Rule $(\geq 2, \geq 1, \geq 0)$**.** Let a single w be a child of v. Branch on v. In the *delete* branch w becomes a tent, in the *fix* branch proceed as in the $(\geq 4, 0, 0)$ rule for w.

Branching Rule $(\geq 2, \geq 0, \geq 1)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(\geq 2, \geq 1, \geq 0)$ rule. In the remaining branches, do not do anything more.

Branching Rule $(1, 0, 1)$**.** Let a double w be the only child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(1, 1, 0)$-rule, in the remaining branches observe that v becomes a D-leaf of U-degree 2, and apply Rule 4 to eliminate it.

Branching Rule $(1, \geq 2, \geq 0)$**.** Let singles w_1, w_2 be children of v. Branch on v. In the *delete* branch w_1 and w_2 become tents. In the 'remove' branch, eliminate both w_1 and w_2 with the $(\geq 4, 0, 0)$ rule.

Branching Rule $(1, \geq 1, \geq 1)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(1, \geq 2, \geq 0)$ rule, in the remaining branches proceed as in the $(\geq 2, \geq 1, \geq 0)$ rule.

Branching Rule $(1, \geq 0, \geq 2)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(1, \geq 1, \geq 1)$ rule, in the remaining branches proceed as in the $(\geq 2, \geq 0, \geq 1)$ rule.

Branching Rule $(0, 1, 1)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch do not do anything more, in the remaining branches proceed as in the $(1, 1, 0)$ rule.

Branching Rule $(0, 0, 2)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(0, 1, 1)$ rule, in the remaining branches proceed as in the $(1, 0, 1)$ rule.

Branching Rule $(0, \geq 3, \geq 0)$**.** Let a single w be a child of v. Branch on w. In the *delete* branch do not do anything more, in the *fix* branch proceed as in $(1, \geq 2, \geq 0)$ rule.

Branching Rule $(0, \geq 2, \geq 1)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(0, \geq 3, \geq 0)$ rule, in the remaining branches proceed as in the $(1, \geq 2, \geq 0)$ rule.

Branching Rule $(0, \geq 1, \geq 2)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(0, \geq 2, \geq 1)$ rule, in the remaining branches proceed as in the $(1, \geq 1, \geq 1)$ rule.

Branching Rule $(0, \geq 0, \geq 3)$**.** Let a double w be a child of v. Use the subroutine to eliminate w. In the *delete* branch proceed as in the $(0, \geq 1, \geq 2)$ rule, in the remaining branches proceed as in the $(1, \geq 0, \geq 2)$ rule.

It is easy to see that for each guide some reduction rule is applicable. Moreover, the branch tree for each rules is of constant size. The rules are built of elementary operations which decrease D, so we obtain a correct algorithm solving the Disjoint-FVS problem.

3.3 Complexity analysis

Due to numerous branching rules, some of them rather complicated, a manual complexity analysis would be tedious and error-prone. Therefore in the Appendix we provide a Python script, which automates the analysis.

Our script computes the *branching vectors*[3]. It relies on a description of all branching rules, strictly following their definitions presented above. Computing the branching vectors, we use the following facts, all proved above.

(a) Rules 1 and 2 do not increase the measure.

(b) Rule 4 decreases the measure by at least $1 - \alpha$.

(c) Obtaining a tent corresponds to drop in measure equal to 1.

(d) Elementary branch on a vertex v with U-degree f has measure drop at least 1 in the *delete* branch and $(f - 1)\alpha$ in the *fix* branch.

Having computed the branching vectors, our script determines β_α: the maximum positive root of the corresponding equations over all vectors. Standard reasoning for branching algorithms lets us conclude that our algorithm for Disjoint-FVS works in $\mathcal{O}^*(\beta_\alpha^{\mu_\alpha(I)})$ time.

The instances of Disjoint-FVS arising from iterative compression of FVS have $|U| = k + 1$, so $\ell' \leq k + 1$. Thus, for such an instance I we have $\mu_\alpha(I) = k(I) + \alpha\ell'(I) - t(I) \leq k(1 + \alpha) + 1$. Consequently, our algorithm solves them in $\mathcal{O}^*\left((\beta_\alpha)^{(1+\alpha)k}\right)$ time and polynomial space. For $\alpha = 0.84$ this becomes $\mathcal{O}^*(2.592^k)$. Repeating the reasoning from the proof of Theorem 2, we complete the proof of Theorem 1.

4 Concluding remarks

In our paper we presented a two deterministic FPT algorithms for Feedback Vertex Set. The former can be seen as a reinterpretation and simplification of the previously fastest algorithm of Cao et al. [6]. The latter algorithm performs branches more carefully and consequently its running time is slightly better.

However, we are still far from matching the time complexity of the best *randomized* algorithm, using the Cut&Count technique [12]. In particular, in our work we do not use any insights both from the

[3] We refer to [17] for more on branching vectors.

Cut&Count technique and its later derandomizations [11, 3]. Obtaining a *deterministic* algorithm for FVS running in $\mathcal{O}^*(3^k)$ time remains a challenging open problem.

We would also like to note that we are not aware of any lower bounds for FPT algorithms of FVS that are stronger than a refutation of a subexponential algorithm, based on the Exponential Time Hypothesis, that follows directly from a similar result for Vertex Cover[4]. Can we show some limits for FPT algorithms for FVS, assuming the Strong Exponential Time Hypothesis, as it was done for e.g. Steiner Tree or Connected Vertex Cover [10]?

References

[1] A. Becker, R. Bar-Yehuda, and D. Geiger. Randomized algorithms for the loop cutset problem. *J. Artif. Intell. Res. (JAIR)*, 12:219–234, 2000.

[2] H. L. Bodlaender. On disjoint cycles. *Int. J. Found. Comput. Sci.*, 5(1):59–68, 1994.

[3] H. L. Bodlaender, M. Cygan, S. Kratsch, and J. Nederlof. Solving weighted and counting variants of connectivity problems parameterized by treewidth deterministically in single exponential time. *CoRR*, abs/1211.1505, 2012.

[4] H. L. Bodlaender and T. C. van Dijk. A cubic kernel for feedback vertex set and loop cutset. *Theory Comput. Syst.*, 46(3):566–597, 2010.

[5] K. Burrage, V. Estivill-Castro, M. R. Fellows, M. A. Langston, S. Mac, and F. A. Rosamond. The undirected feedback vertex set problem has a poly() kernel. In H. L. Bodlaender and M. A. Langston, editors, *IWPEC*, volume 4169 of *Lecture Notes in Computer Science*, pages 192–202. Springer, 2006.

[6] Y. Cao, J. Chen, and Y. Liu. On feedback vertex set new measure and new structures. In H. Kaplan, editor, *SWAT*, volume 6139 of *Lecture Notes in Computer Science*, pages 93–104. Springer, 2010.

[7] J. Chen, F. V. Fomin, Y. Liu, S. Lu, and Y. Villanger. Improved algorithms for feedback vertex set problems. *J. Comput. Syst. Sci.*, 74(7):1188–1198, 2008.

[8] J. Chen, Y. Liu, S. Lu, B. O'Sullivan, and I. Razgon. A fixed-parameter algorithm for the directed feedback vertex set problem. *J. ACM*, 55(5), 2008.

[9] R. H. Chitnis, M. Cygan, M. T. Hajiaghayi, and D. Marx. Directed subset feedback vertex set is fixed-parameter tractable. In A. Czumaj, K. Mehlhorn, A. M. Pitts, and R. Wattenhofer, editors, *ICALP (1)*, volume 7391 of *Lecture Notes in Computer Science*, pages 230–241. Springer, 2012.

[10] M. Cygan, H. Dell, D. Lokshtanov, D. Marx, J. Nederlof, Y. Okamoto, R. Paturi, S. Saurabh, and M. Wahlström. On problems as hard as cnf-sat. In *IEEE Conference on Computational Complexity*, pages 74–84. IEEE, 2012.

[11] M. Cygan, S. Kratsch, and J. Nederlof. Fast hamiltonicity checking via bases of perfect matchings. *CoRR*, abs/1211.1506, 2012.

[12] M. Cygan, J. Nederlof, M. Pilipczuk, M. Pilipczuk, J. M. M. van Rooij, and J. O. Wojtaszczyk. Solving connectivity problems parameterized by treewidth in single exponential time. In R. Ostrovsky, editor, *FOCS*, pages 150–159. IEEE, 2011.

[13] M. Cygan, M. Pilipczuk, M. Pilipczuk, and J. O. Wojtaszczyk. Subset feedback vertex set is fixed-parameter tractable. *SIAM J. Discrete Math.*, 27(1):290–309, 2013.

[14] F. K. H. A. Dehne, M. R. Fellows, M. A. Langston, F. A. Rosamond, and K. Stevens. An $O(2^{O(k)})n^3$ FPT algorithm for the undirected feedback vertex set problem. *Theory Comput. Syst.*, 41(3):479–492, 2007.

[15] R. G. Downey and M. R. Fellows. Fixed parameter tractability and completeness. In *Complexity Theory: Current Research*, pages 191–225, 1992.

[16] R. G. Downey and M. R. Fellows. *Parameterized Complexity*. Springer, 1999.

[17] F. Fomin and D. Kratsch. *Exact Exponential Algorithms*. Texts in theoretical computer science. Springer Berlin Heidelberg, 2010.

[18] H. N. Gabow and M. F. M. Stallmann. Efficient algorithms for graphic matroid intersection and parity (extended abstract). In W. Brauer, editor, *ICALP*, volume 194 of *Lecture Notes in Computer Science*, pages 210–220. Springer, 1985.

[19] J. Guo, J. Gramm, F. Hüffner, R. Niedermeier, and S. Wernicke. Compression-based fixed-parameter algorithms for feedback vertex set and edge bipartization. *J. Comput. Syst. Sci.*, 72(8):1386–1396, 2006.

[20] I. A. Kanj, M. J. Pelsmajer, and M. Schaefer. Parameterized algorithms for feedback vertex set. In R. G. Downey, M. R. Fellows, and F. K. H. A. Dehne, editors, *IWPEC*, volume 3162 of *Lecture Notes in Computer Science*, pages 235–247. Springer, 2004.

[4]Replace each edge with a short cycle to obtain a reduction from Vertex Cover to Feedback Vertex Set.

[21] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, The IBM Research Symposia Series, pages 85–103. Plenum Press, New York, 1972.

[22] J. Oxley. *Matroid Theory*. Oxford University Press, 2 edition, 2011.

[23] V. Raman, S. Saurabh, and C. R. Subramanian. Faster fixed parameter tractable algorithms for finding feedback vertex sets. *ACM Transactions on Algorithms*, 2(3):403–415, 2006.

[24] B. A. Reed, K. Smith, and A. Vetta. Finding odd cycle transversals. *Oper. Res. Lett.*, 32(4):299–301, 2004.

[25] S. Thomassé. A $4k^2$ kernel for feedback vertex set. *ACM Transactions on Algorithms*, 6(2):32:1–32:8, 2010.

Python script automating complexity analysis[5]

```
import scipy.optimize

def value(vector):
    """compute the value of a branching vector"""
    def h(x):
        return sum([x**(-v) for v in vector])-1
    return scipy.optimize.brenth(h,1, 100)


def join(first, then):
     """peform 'then' in each branch after the execution of 'first' """
     return [x+y for x in first for y in then]


alpha = 0.84

no   = [0]        # trivial continuation, do not do anything more
zero = [0]        # perform reduction rule 1
one  = [0]        # perform reduction rule 2
two  = [1-alpha]  # perform reduction rule 4
tent = [1]        # discover a tent

def br (f, delete, fix):
    """perform an  elementary branch on a vertex of U-degree f"""
    return join([1], delete) + join([alpha*(f-1)], fix)

guide = dict()
guide[('1','1','0')] = br(3, one, two)
guide[('>=4','0','0')] = br(4, no, no)

def double(delete, fix):
    """perform the double elimination subroutine"""
    return br(3, delete, br(3, join(one, fix), join(two, fix)))

guide[('>=2','>=1','>=0')] = br(2, tent, guide[('>=4','0','0')])
guide[('>=2','>=0','>=1')] = double(guide[('>=2','>=1','>=0')], no)
guide[('1','0','1')]       = double(guide[('1','1','0')], two)
guide[('1','>=2','>=0')]   = br(1, join(tent, tent), join(guide[('>=4','0','0')],guide[('>=4','0','0')]))
guide[('1','>=1','>=1')]   = double(guide[('1','>=2','>=0')], guide[('>=2','>=1','>=0')])
guide[('1','>=0','>=2')]   = double(guide[('1','>=1','>=1')], guide[('>=2','>=0','>=1')])
guide[('0','1','1')]       = double(no, guide[('1','1','0')])
guide[('0','0','2')]       = double(guide[('0','1','1')] , guide[('1','0','1')])
guide[('0','>=3','>=0')]   = br(3,no, guide[('1','>=2','>=0')])
guide[('0','>=2','>=1')]   = double(guide[('0','>=3','>=0')], guide[('1','>=2','>=0')])
guide[('0','>=1','>=2')]   = double(guide[('0','>=2','>=1')], guide[('1','>=1','>=1')])
guide[('0','>=0','>=3')]   = double(guide[('0','>=1','>=2')], guide[('1','>=0','>=2')])


vectors = guide.values()

print max([value(b) for b in vectors])**(1+alpha)
```

[5] Also available at `students.mimuw.edu.pl/~kociumaka/fvs`